

Лабораторная 12. Нагрузочное тестирование KnowBase

Цель работы

Научиться запускать простой нагрузочный тест API в k6, сравнивать поведение системы при 10, 20 и 50 виртуальных пользователях, фиксировать RPS, p95 и долю ошибок, а затем делать вывод о наличии или отсутствии деградации.

Результат работы

- 1. установлен k6 и подтверждена его работоспособность
- 2. подготовлен один понятный скрипт для поиска и RAG-чата
- 3. выполнены три разрешённых запуска на 10, 20 и 50 VU
- 4. собраны RPS, p95 и error rate для каждого запуска
- 5. сформулирован краткий вывод о деградации системы

Главное правило безопасности. Запускать скрипт можно только после того, как преподаватель выдаст адрес отдельного учебного стенда, токен и разрешённое окно. Запрещено указывать production, общий тестовый контур, публичный сайт или произвольный адрес. Если хотя бы одного условия нет, студент выполняет подготовительную часть и не запускает нагрузку.

Список ссылок

Адрес учебного стенда	Чтобы k6 знал единственный разрешённый BASE_URL.
Swagger	Чтобы корректно указать методы, пути, заголовки и JSON-тела.
Учебный токен	Чтобы не размещать постоянные учётные данные в скрипте.
Тестовые данные	Чтобы запросы были повторяемыми и не затрагивали реальные документы.
Режим LAB / MOCK для чата	Чтобы RAG-чат не обращался к платной модели и не создавал тяжёлую очередь генерации.

Окно запуска и ответственный	Чтобы группы не запускали 50 VU одновременно и знали, кому сообщить о проблеме.
Наблюдение за стендом	Чтобы преподаватель видел ошибки, ресурсы и мог немедленно остановить тест.

1. Установка k6

k6 распространяется бесплатно. Выберите один способ установки для своей операционной системы. После установки откройте терминал или PowerShell и выполните команду `k6 version`. В ответ должна появиться версия программы.

Windows	<code>winget install k6 --source winget</code> , либо официальный MSI-установщик
macOS	<code>brew install k6</code>
Ubuntu / Debian	Установка из официального репозитория k6 по инструкции Grafana
Любая система с Docker	<code>docker pull grafana/k6</code>

Официальная инструкция и ссылки на установочные файлы: [Install k6](#)

2. Подготовка файлов

1. Создайте отдельную папку `lab12_k`
2. Создайте в ней файл `knowbase_load.js`.
3. Скопируйте в файл шаблон скрипта из следующего раздела.
4. Не заменяйте адрес, пути API и поля JSON самостоятельно. Используйте подготовленный преподавателем вариант после сверки со Swagger.

3. Шаблон скрипта k6

Шаблон создаёт одинаковый короткий сценарий. Виртуальный пользователь отправляет запрос поиска, затем вопрос в RAG-чат и делает паузу. Запросы помечены тегами `search` и `chat`, поэтому k6 покажет отдельные показатели для двух операций.

```

import http from 'k6/http';
import { check, sleep } from 'k6';

const VUS = Number(__ENV.VUS || 0);
const BASE_URL = __ENV.BASE_URL || "";
const LAB_TOKEN = __ENV.LAB_TOKEN || "";
const CONFIRM = __ENV.CONFIRM_LAB_RUN || "";

// Преподаватель заменяет значение перед выдачей файла.
const ALLOWED_BASE_URL =
'https://ТУТ-УКАЗАТЬ-УЧЕБНЫЙ-СТЕНД';
const ALLOWED_VUS = [10, 20, 50];

if (!ALLOWED_VUS.includes(VUS)) {
  throw new Error('Разрешены только 10, 20 или 50 VU');
}
if (BASE_URL !== ALLOWED_BASE_URL) {
  throw new Error('BASE_URL не совпадает с разрешённым стендом');
}
if (!LAB_TOKEN || CONFIRM !== 'YES') {
  throw new Error('Нет токена или подтверждения запуска');
}

export const options = {
  vus: VUS,
  duration: '30s',
  gracefulStop: '5s',
  thresholds: {
    http_req_failed: [{
      threshold: 'rate<0.10',
      abortOnFail: true,
      delayAbortEval: '10s',
    }],
    http_req_duration: [{
      threshold: 'p(95)<10000',
      abortOnFail: true,
      delayAbortEval: '10s',
    }],
    'http_reqs{endpoint:search}': ['count>0'],
  },
};

```

```

    'http_reqs{endpoint:chat}': ['count>0'],
    'http_req_duration{endpoint:search}': ['p(95)<10000'],
    'http_req_duration{endpoint:chat}': ['p(95)<10000'],
    'http_req_failed{endpoint:search}': ['rate<0.10'],
    'http_req_failed{endpoint:chat}': ['rate<0.10'],
  },
};

const params = {
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${LAB_TOKEN}`,
    'X-Lab-Mode': 'mock',
  },
};

export default function () {
  const searchResponse = http.post(
    `${BASE_URL}/api/search`,
    JSON.stringify({ query: 'нагрузочное тестирование',
limit: 5 })),
    { ...params, tags: { endpoint: 'search' } },
  );
  check(searchResponse, {
    'search: status 200': (r) => r.status === 200,
  });

  sleep(1);

  const chatResponse = http.post(
    `${BASE_URL}/api/chat`,
    JSON.stringify({
      question: 'Что такое нагрузочное тестирование?',
      mode: 'mock',
    }),
    { ...params, tags: { endpoint: 'chat' } },
  );
  check(chatResponse, {
    'chat: status 200': (r) => r.status === 200,
  });

  sleep(1);
}

```

Здесь преподавателю заменить ALLOWED_BASE_URL на точный адрес учебного стенда.

Здесь преподавателю сверить пути /api/search и /api/chat со Swagger.

Здесь преподавателю заменить JSON-тела и ожидаемые статусы на фактический контракт API.

Здесь преподавателю подтвердить, что заголовок X-Lab-Mode: mock действительно включает безопасную заглушку RAG.

4. Проверочный запуск

Проверочный запуск выполняет преподаватель на 1 VU до начала занятия. Студентам режим 1 VU в итоговом файле можно не оставлять. Преподаватель убеждается, что оба запроса возвращают ожидаемый статус, в логах нет побочных операций, а чат работает через заглушку.

Здесь добавить подтверждение преподавателя: «Скрипт проверен на 1 VU, дата, ФИО».

5. Получение разрешения на запуск

Перед каждым запуском студент проверяет все пункты:

1. адрес совпадает с адресом учебного стенда, выданным преподавателем
2. получен временный LAB_TOKEN
3. наступило разрешённое окно запуска
4. преподаватель разрешил текущей группе начать тест
5. предыдущая группа завершила работу
6. открыт канал связи для команды «СТОП»

Команда «СТОП». Немедленно остановите k6 сочетанием Ctrl+C, если преподаватель дал команду, error rate превысил 10%, появились массовые ответы 429 или 5xx, стенд перестал отвечать либо возникло необычно долгое ожидание. После остановки не повторяйте запуск без нового разрешения.

6. Три запуска нагрузки

Каждый запуск длится 30 секунд. Между запусками выдерживается пауза не менее 3 минут. Одновременно тест выполняет только один студент или одна группа. Команды ниже являются шаблонами. Подставлять можно только значения, выданные преподавателем.

Запуск 1. 10 VU

```
k6 run -e BASE_URL="АДРЕС_ИЗ_ЗАДАНИЯ" -e LAB_TOKEN="ВРЕМЕННЫЙ_TOKEN" -e  
CONFIRM_LAB_RUN=YES -e VUS=10 knowbase_load.js
```

Запуск 2. 20 VU

```
k6 run -e BASE_URL="АДРЕС_ИЗ_ЗАДАНИЯ" -e LAB_TOKEN="ВРЕМЕННЫЙ_TOKEN" -e  
CONFIRM_LAB_RUN=YES -e VUS=20 knowbase_load.js
```

Запуск 3. 50 VU

```
k6 run -e BASE_URL="АДРЕС_ИЗ_ЗАДАНИЯ" -e LAB_TOKEN="ВРЕМЕННЫЙ_ТОКЕН" -e
CONFIRM_LAB_RUN=YES -e VUS=50 knowbase_load.js
```

7. Какие значения записать

После каждого запуска найдите в итоговом отчёте строки для search и chat. Зафиксируйте RPS, p95 и errorrate. RPS показывает среднее количество запросов в секунду, p95 показывает время, быстрее которого завершилось не менее 95% запросов, error rate показывает долю неуспешных HTTP-запросов.

10	Поиск				
10	RAG-чат				
20	Поиск				
20	RAG-чат				
50	Поиск				
50	RAG-чат				

8. Как оценить деградацию

Сначала сравните 20 VU с 10 VU, затем 50 VU с 10 VU. Рассчитайте рост p95 по формуле: $((p95 \text{ при новой нагрузке} - p95 \text{ при } 10 \text{ VU}) / p95 \text{ при } 10 \text{ VU}) \times 100\%$. Для error rate сравните разницу в процентных пунктах.

Стабильное поведение	p95 меняется незначительно, error rate остаётся близким к исходному, RPSувеличивается при росте VU.
Заметная деградация	p95 растёт, ошибки появляются чаще, а прирост RPS становится намного меньше прироста VU.

Критическая деградация	RPS перестаёт расти или снижается, p95 резко увеличивается, error gate превышает 5% либо тест аварийно остановлен.
------------------------	--

Эти признаки применяются только для учебного анализа. Они не заменяют требования к производительности, SLA или SLO реального проекта. Если проектные пороги будут утверждены, вывод нужно делать по ним.

9. Вопросы для вывода

1. Как менялся RPS при переходе от 10 к 20 и 50 VU?
2. Как менялся p95 отдельно для поиска и RAG-чата?
3. При какой нагрузке появились или выросли ошибки?
4. Есть ли точка, после которой увеличение VU почти не даёт прироста RPS?
5. Наблюдается ли деградация и какими числами это подтверждается?

Структура отчёта студента

1. **Титульный лист.** Университет, дисциплина, номер и название работы, ФИО, группа, дата.
2. **Подготовка.** Версия k6 и подтверждение успешной установки.
3. **Скрипт.** Файл knowbase_load.js без токена и других секретов.
4. **Результаты.** Заполненная таблица для 10, 20 и 50 VU.
5. **Доказательства.** По одному скриншоту итогового отчёта каждого запуска.
6. **Вывод.** 5-8 предложений с опорой на RPS, p95 и error rate.

Отчёт сдаётся в редактируемом формате docx. LAB_TOKEN в отчёт не включается.

Критерии оценивания

k6 установлен, скрипт подготовлен и соответствует выданному шаблону.	1 балл
Корректно выполнены разрешённые запуски на 10, 20 и 50 VU.	1,5 балла
Верно зафиксированы RPS, p95 и error rate для поиска и RAG-чата.	1,5 балла
Сделан обоснованный вывод о деградации, отчёт полный и аккуратно оформлен.	1 балл
Итого	5 баллов